Kubron

Argo CD Module: User Guide & Developer Documentation

Version 0.2.0 · Wails + Vue + Go · October 2026

Part I. User Guide

1. Overview

The Argo CD module is built into Kubron - no extra plugins. If the active cluster has the Application CRD (argoproj.io/v1alpha1), the Argo CD section appears in the menu.

Key capabilities:

- Applications - sync/health, bulk actions, live updates
- ApplicationSets, Projects, Repositories, Clusters (Argo settings)
- Sync (prune / force / dry-run / revision), refresh / hard refresh, terminate
- Suspend / resume auto-sync, history rollback, delete cascade|keep
- Resource graph and live-vs-desired diff (via Argo CD API + port-forward)
- Open the official Argo CD UI via port-forward
- Dashboard: stats, apps needing attention, recent syncs

ℹ️ Most operations use the Kubernetes API (dynamic client). Diff and API token generation need a local port-forward to argocd-server and an Argo CD account token.

2. Navigation

The section is visible only when ArgoInstalled() is true. Subsections:

- Dashboard - applications summary
- Applications - Application list
- ApplicationSets - ApplicationSet list
- Projects - AppProject list
- Repositories / Clusters - configuration (Argo Secrets)

3. Dashboard

Cards: total apps, synced / out-of-sync, healthy / degraded / missing / progressing / suspended. Blocks for apps needing attention, recent syncs, grouping by project and destination cluster. Actions: Refresh, Sync, Open Argo UI.

4. Applications

List columns: name, namespace, project, sync, health, repo, path/chart, destination, last sync, age. Live updates via the argocd:changed event (informer).

4.1. Application detail

- Properties: sync/health, policy (manual | automated + prune + self-heal), revision, operation in progress

- Tabs: Resources, Graph, Diff, Sources, History
- Conditions - Application conditions

4.2. Actions

- Sync - dialog: optional revision, prune, force, dry-run; bulk for multiple apps
- Refresh / Hard refresh - recompute (hard clears the manifest cache)
- Terminate operation - stop the running operation
- Enable / Disable auto-sync
- Rollback - to a history entry (unavailable while auto-sync is on)
- Delete - cascade (with resources) or keep (Application object only)
- Open Argo UI - port-forward to argocd-server

⚠️ Force uses replace/delete-recreate and may cause downtime. Prune deletes resources marked requiresPruning.

5. Sync

For a single app you can set revision (empty = targetRevision from the spec). Bulk sync does not accept a revision. Dry-run shows what would change without applying. Resources that would be pruned are listed before confirm.

6. Resources, Graph, Diff

Resources

Resources from status.resources[]: kind, name, namespace, sync, health, hook, requiresPruning. Filter for problems only.

Graph

Graph: managed resources plus related objects (ReplicaSet, Pod, Service←Ingress). Edges: owns / selects / routes. Filter by kind and problems-only; collapsible groups.

Diff

Live vs desired via the Argo CD REST API (managed-resources). Requires: (1) an Argo CD API token for the cluster, (2) port-forward to argocd-server. Secret values are redacted. The list may be truncated (limit 1000 resources).

ℹ️ Configure the token in the Diff UI: paste an existing one or generate (account + password, lifetime in days). Stored locally under ~/.kubron/argocd (mode 0600); never sent to the frontend.

7. History and rollback

status.history[]: id, revision, deployedAt, source. Rollback starts a sync to the selected entry. With auto-sync enabled, rollback is blocked - disable auto-sync first.

8. ApplicationSets

ApplicationSet list: generators, app count, project, repo. Applications tab lists generated Applications. Preserve-on-deletion and applicationsSync appear in properties.

9. Projects

AppProject: sourceRepos, destinations (cluster + namespace), whitelist/blacklist for cluster and namespaced resources, roles (Access tab). Roles and access policies are view-only.

10. Repositories and Clusters

Read from Argo CD Secrets (not the same as Kubron Helm repositories). Shows URL, auth type, insecure TLS, proxy - never secret values.

11. Open Argo UI

Finds the argocd-server Service, starts a port-forward, opens http(s)://127.0.0.1:<port>. Repeat calls reuse the active tunnel. Uses http when server.insecure=true in argocd-cmd-params-cm.

12. Permissions

You need RBAC on Application / ApplicationSet / AppProject and related Secrets. Sync/delete/terminate need write. Diff needs an Argo CD API token (separate from kubeconfig). Local Kubron roles (admin/operator/user) further restrict the UI.

13. Limitations in 0.2.0

- No Application creation from the UI (manage existing only)
- No editing of Project roles / repo credentials
- Diff depends on argocd-server reachability and a token
- Helm values / parameters in Sources are omitted (secrets)
- Rollback unavailable while auto-sync is enabled

Part II. Developer Documentation

14. Architecture

Two channels: (A) Kubernetes API - list/watch CRDs, patch for sync/refresh/terminate/rollback/delete; (B) Argo CD REST API via port-forward - managed-resources diff and token generation.

14.1. Layers

Layer	Responsibility
Frontend (Vue 3)	Views, tabs, composables, resource defs, i18n
Wails bridge	callApp → Manager / PortForwardManager
internal/k8s argocd_*	CRD access, actions, graph, settings, cache
Port-forward + REST	Diff, token gen, Open Argo UI
Local FS	~/.kubron/argocd - API tokens (0600)

14.2. Backend files

File	Purpose

argocd.go	ArgoInstalled - discovery CRD applications
argocd_types.go	DTO: Application, ResourceRef, History, Source, ...
argocd_apps.go	GetArgoApplications
argocd_actions.go	Sync, Refresh, Terminate, Rollback, AutoSync, Delete
argocd_sets.go	GetArgoApplicationSets
argocd_projects.go	GetArgoProjects (+ roles/destinations)
argocd_settings.go	GetArgoRepositories, GetArgoClusters
argocd_graph.go	GetArgoApplicationGraph
argocd_diff.go	GetArgoApplicationDiff (PortForwardManager)
argocd_api.go	Minimal REST client to argocd-server
argocd_token*.go	Store / generate / status API token
argocd_ui.go	OpenArgoUI - port-forward to UI
argocd_cache.go	Informer → event argocd:changed

14.3. Frontend

- `components/views/ArgoDashboardView.vue`
- `components/resource/sections/Argo*.vue` - Diff, Graph, History, Resources, Sources, ProjectAccess/Roles, SetApps
- `resources/definitions/argocd/` - application, applicationset, project, repository, cluster
- `composables/useArgo.ts` - ArgoInstalled visibility
- `composables/useArgoLive.ts` - argoLiveTick on argocd:changed
- `composables/useArgoApplications.ts` - dashboard list + flash on change
- `i18n/locales/{en,ru}/argo.json`

15. Data model

ArgoApplicationInfo

name, namespace, project, age, syncStatus, healthStatus, revision, repoURL, path, chart, targetRevision, destServer/Name/Namespace, autoSync, selfHeal, prune, operationOn, ...

ArgoResourceRef

group, version, kind, namespace, name, status, health, message, hook, requiresPruning

ArgoSyncOptions / ArgoAppRef / ArgoActionResult

revision, prune, force, dryRun; namespace+name; per-app error string

ArgoGraph / ArgoGraphNode / ArgoGraphEdge

nodes with managed flag; edges owns | selects | routes

ArgoDiff / ArgoResourceDiff

state InSync|Modified|..., unified diff text, truncated flag

16. Actions (Kubernetes API)

Implemented via patch/update of Application (operation, syncPolicy, finalizers). Timeout 60s, bulk ≤ 200 apps. Checks: not deleting, no concurrent operation (errArgoBusy).

- SyncArgoApplication(apps, opts)

- RefreshArgoApplication(apps, hard)
- TerminateArgoOperation(apps)
- RollbackArgoApplication(ns, name, id, prune, dryRun)
- SuspendArgoAutoSync / ResumeArgoAutoSync
- DeleteArgoApplication(ns, name, cascade)

17. Diff and API token

GetArgoApplicationDiff on PortForwardManager: starts a tunnel, calls REST managed-resources, builds unified diff (LCS, line/byte limits). Secret data redacted. TLS InsecureSkipVerify only for loopback.

Token: SetArgoAPIToken / DeleteArgoAPIToken / GetArgoAPIStatus. File: hash(clusterID) under ~/.kubron/argocd, mode 0600. Never returned to the frontend, never logged or audited as a value. GenerateArgoAPIToken - session login via port-forward, then save.

18. Live updates

argocd_cache.go: informer on Application. EventsEmit argocd:changed → useArgoLive.argoLiveTick++ → resource definitions refreshOn / dashboard reload. Without list/watch rights the informer does not start; autoRefresh remains as fallback.

19. Security

- Sanitization: repoURL without userinfo; Helm values/parameters never returned
- Diff: redact Secrets; Authorization header does not follow redirects
- Token only on disk 0600; status is a boolean configured flag
- Port-forward to localhost only; insecure TLS only on loopback
- Limits: bulk 200, API body 64 MiB, diff resources 1000, lines 2000

20. Adding an action

1. Backend: dynamic-client function + Manager wrapper, timeout, bulk safety
2. Do not log or return secrets
3. Frontend: argoActions helper + menu item + i18n en/ru
4. Test against a fake dynamic client

21. Tests

- Discovery without CRD → ArgoInstalled false
- Sync/refresh/terminate/rollback patch semantics
- Busy / deleting application errors
- Token path isolation, normalize rejects bad tokens
- Diff redact + truncation
- Graph edge types owns/selects/routes

22. Related areas

- Helm - separate module; do not mix Helm releases with Applications
- Port-forward subsystem - shared with Pod port-forward
- Audit log - who ran sync/delete
- pkg/ext - optional external token store (ErrNotFound)

This document is based on Kubron v0.2.0 source and UI (https://github.com/VadimIl224/kubron). If it disagrees with the code, the repository wins.